

HelixConstitution

HelixConstitution

La constitución universal de ingeniería que hereda cada proyecto — ley anti-engaño, aplicada mecánicamente, compartida como un submódulo de Git.

Resumen

HelixConstitution es el único reglamento agnóstico a los proyectos —incorporado como submódulo de Git en cada proyecto Helix/vasic-digital— que codifica una disciplina de ingeniería innegociable (anti-engaño, validación basada únicamente en evidencia, seguridad de datos y hosts, documentación y cobertura de pruebas) y la propaga a una flota de más de 140 repositorios. Es la columna vertebral de gobernanza que dota de coherencia a todo el ecosistema.

Descripción breve

Un Constitution universal e heredable, distribuido como submódulo de Git. Establece reglas obligatorias e innegociables —barreras de evidencia anti-engaño, inmunidad a falsos positivos, seguridad de datos y hosts, disciplina en cobertura y documentación— que cada proyecto que lo adopta hereda automáticamente y puede extender, pero nunca debilitar.

Descripción detallada

HelixConstitution es la fuente canónica y única de verdad para las prácticas de ingeniería compartidas por todos los proyectos que deciden adoptarlo como submódulo de Git —ley de ingeniería distribuida y versionada exactamente como el código—. Su pieza central —Constitution.md— es un documento de ~1 MB, versionado de forma continua, compuesto por cláusulas numeradas (la familia de pactos §11.4.x, actualmente hasta §11.4.170) y manuales operativos por agente (CLAUDE.md, AGENTS.md,

QWEN.md, GEMINI.md), que lo importan por referencia para que tanto humanos como cada agente CLI consulten un mismo reglamento idéntico. La herencia está diseñada en tres capas: la base universal (este submódulo), la capa del proyecto (el propio Constitution/CLAUDE/AGENTS del proyecto que lo extiende) y una capa opcional por subdirectorío —evaluadas de arriba hacia abajo, donde un proyecto puede *endurecer* las reglas, pero está arquitectónicamente impedido de *relajarlas*—. El resultado es una flota de más de 140 repositorios que no pueden divergir en silencio, pues la disciplina que comparten está fijada, no memorizada.

El documento es intransigentemente agnóstico al dominio: cualquier mención a un proveedor específico, SKU de hardware, puerto o versión de biblioteca debe trasladarse al propio Constitution del proyecto que lo consume, y la universalidad nunca se da por sentada —debe *ganarse* superando una prueba explícita de cuatro partes antes de que una regla sea admitida en la base—. Su columna vertebral filosófica es el anti-engaño, expresado como una familia interconectada de pactos —§1.1 inmunidad a falsos positivos, §11.4 pacto de calidad para el usuario final, §11.4.6 prohibición de suposiciones, §11.4.69 taxonomía de evidencia positiva— cuyo efecto combinado traza una línea roja única: el listón para lanzar un producto nunca es “las pruebas pasan”, sino “un usuario real puede usar la función”, y todo resultado positivo debe respaldarse con evidencia física capturada o no cuenta. Un documento complementario, submodules-catalogue.md (142 repositorios), convierte la pregunta “¿ya tenemos algo que haga esto?” en un reflejo de catálogo primero, extender en lugar de reimplementar, antes de escribir una sola línea de código nuevo. Scripts auxiliares localizan el submódulo desde cualquier nivel de anidamiento y difunden cada commit a cuatro proveedores de Git independientes, de modo que el reglamento autoritativo único también resulta imposible de perder.

Por qué lo creamos

Múltiples aplicaciones de producto grandes y docenas de submódulos reutilizables desacoplados, creados por el mismo propietario, seguían redescubriendo las mismas reglas ganadas con esfuerzo —y seguían topándose con la misma clase de fallo: pruebas e informes de estado que afirmaban éxito mientras la función estaba rota para el usuario final (“falsos positivos” y “falsos negativos”). Cada anclaje forense en el Constitution registra un incidente real (por ejemplo, el falso positivo del enrutamiento de audio D3 del 20-05-2026, donde la validación se puso en verde con el campo “Códec en uso” vacío, o la interfaz de botón gigante del 25-06-2026 que superó las pruebas de igualdad de tokens mientras la pantalla real estaba rota). El Constitution

existe para hacer mecánicamente imposible, de una vez y para siempre, toda esa clase de éxito deshonesto, de modo que la disciplina no se diluya entre proyectos ni caiga en el olvido.

Por qué es un cambio radical

Transforma la cultura de ingeniería de un conjunto de documentación que la gente espera seguir en una ley heredada, versionada y aplicada mecánicamente —la diferencia entre una guía de estilo y un compilador—. Una actualización de submódulo actualiza las reglas para toda la flota de una vez, de forma atómica y trazable. Un único pacto anti-falsos positivos está *garantizado* presente en cada repositorio consumidor, no por confianza, sino por diseño: una puerta de propagación busca literalmente el número de cláusula en toda la flota, y una prueba de mutación asociada demuestra que la puerta en sí no miente —así que incluso la aplicación de las normas está aplicada—. La gobernanza deja de ser una aspiración en una wiki que nadie lee y se convierte en un hecho auditable y comprobable al que puede apuntar un trabajo de CI.

Qué tiene de innovador

- **Constitution como submódulo:** la ley de ingeniería se distribuye y versiona exactamente como el código, con etiquetas deliberadas al estilo v1.0.0 y fijación por proyecto, de modo que cada repositorio sabe *exactamente* a qué revisión de la ley está sujeto.
- **Anti-falsos positivos como doctrina forense de primer orden:** cada cláusula se remonta a un mandato literal del operador y, a menudo, al incidente real que la motivó, por lo que el reglamento se lee como jurisprudencia en lugar de opinión.
- **Metapruebas de las propias reglas (§1.1):** cada puerta va acompañada de una mutación que debe cambiar PASS→FAIL, de modo que “la puerta no es un engaño” no se afirma, sino que se demuestra en cada ejecución; una puerta que nunca puede fallar se considera peor que no tener puerta alguna.
- **Universalidad ganada:** un test explícito de cuatro partes decide si una regla es verdaderamente universal o solo específica de un proyecto, manteniendo la base ágil, portable y libre de filtraciones de proveedores.

Cómo se usa en todos los productos (los poderes que otorga)

Como **pilar de gobernanza obligatorio**, el HelixConstitution no es un documento que la familia consulte: es la estructura portante sobre la que se construye la familia.

- **Columna vertebral de gobernanza:** cada proyecto Helix/vasic-digital lo añade como submódulo e lo importa desde CLAUDE.md / AGENTS.md / QWEN.md / su propio Constitution.md; las reglas se aplican sin condiciones, desde el primer commit, sin opción de exclusión por proyecto.
- **Puertas y mandatos:** define el modelo de cobertura de cuatro capas —presencia en el código, supervivencia al build, comportamiento en runtime, puerta-no-falsa— que una función debe superar en los cuatro niveles antes de considerarse terminada, además de un creciente catálogo de mandatos nombrados: manejo de credenciales (§11.4.10), sincronización obligatoria de documentación (§11.4.60), mandato de submódulos en contenedores (§11.4.76), CodeGraph (§11.4.78), cobertura obligatoria de tipos de prueba (§11.4.169), entre otros.
- **Propagación:** las puertas CM-COVENANT-114-NNN-PROPAGATION verifican que el *texto literal* de la cláusula esté presente en toda la flota consumidora, de modo que un pacto no pueda eliminarse discretamente en un rincón del sistema; el incumplimiento es un bloqueo de lanzamiento sin escapatoria ni banderas para ignorarlo.
- **Descubrimiento:** submodules-catalogue.md convierte la pregunta “¿ya tenemos algo que haga X?” en una respuesta de un vistazo antes de crear cualquier nuevo módulo, eliminando el esfuerzo duplicado desde el origen.
- **Consistencia de los agentes AI:** la misma ley se expresa de forma idéntica para cada agente CLI (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi a través de AGENTS.md, Qwen Code mediante QWEN.md), de modo que, sin importar qué herramienta toque el código, obedece uno y el mismo pacto.

Mayores desafíos técnicos y cómo los resolvimos

- **Localizar el submódulo desde una profundidad anidada arbitraria** — una regla enterrada tres submódulos de profundidad debe encontrar la ley sin saber dónde reside → `find_constitution.sh` recorre los directorios padre y sigue el puntero del superproyecto de Git de forma recursiva, respetando un parámetro de anulación `CONSTITUTION_DIR` y dos estructuras admitidas (`constitution/`, `submodules/constitution/`),

de modo que la resolución es determinista sin importar la profundidad del anidamiento.

- **Mantener un repositorio autoritativo en cuatro proveedores de Git** — los espejos no sirven de nada si divergen → `install_upstreams.sh` lee los remotos declarativos `Upstreams/*.sh` y configura origin con múltiples URLs de envío, de manera que un único `git push` se distribuye atómicamente a GitHub (primario), GitLab, GitFlic y GitVerse, sin que ningún espejo quede desactualizado.
- **Evitar la inflación de reglas / filtración de proyectos en la base universal** — cada tentación de “agregarlo aquí” erosiona la portabilidad → se aplica a *cada* nueva regla la prueba de universalidad ganada en cuatro partes, junto con la clasificación universal vs. proyecto del §11.4.17, obligando a que las preocupaciones específicas de un proyecto se mantengan en la capa del proyecto, donde corresponden.
- **Demostrar que el mecanismo de herencia realmente funciona** — un filtro que nunca falla es un filtro en el que no se puede confiar → `meta_test_inheritance.sh`, una meta-prueba centinela, elimina deliberadamente el anclaje del §11.4 y verifica que el filtro lo detecte, de modo que el mecanismo de aplicación se revalida continuamente contra roturas silenciosas.

Pila tecnológica

- **Herencia mediante submódulos de Git** — *por qué*: los submódulos de Git son el único mecanismo que permite que un reglamento sea autoritativo y esté versionado por consumidor, actualizándose mediante un cambio explícito y revisable, en lugar de una copia silenciosa; *cómo*: los proyectos que lo consumen añaden el submódulo y `@import` sus archivos de agente, y las tres capas se evalúan de arriba abajo con un contrato estricto de *extiende-sin-debilitar* en cada frontera.
- **`find_constitution.sh`** — *por qué*: las reglas son inútiles si el código profundamente anidado no puede encontrarlas de manera fiable, y codificar rutas fijas rompería en cuanto un proyecto se reorganizara; *cómo*: un recorrido por directorios padre más recursión con `git rev-parse --show-superproject-working-tree`, respaldado por un parámetro de anulación `CONSTITUTION_DIR`, resolviendo ambas estructuras admitidas.
- **`install_upstreams.sh` + `Upstreams/`** — *por qué*: la redundancia de cuatro proveedores solo es real si no requiere esfuerzo adicional para mantenerse, de lo contrario los espejos se degradan; *cómo*: los archivos `.sh` declarativos por remoto se materializan en un único origin multi-URL, condensando cuatro envíos en uno.
- **Metapruebas de mutación del §1.1** — *por qué*: un filtro que nunca falla es peor que no tener ninguno, porque genera falsa confianza; *cómo*: cada filtro se empareja con una mutación de

eliminación/renombrado mediante sed que debe convertir PASS→FAIL y luego se restaura, de modo que cada filtro demuestra que sigue siendo efectivo en cada ejecución.

- **Filtros de propagación (CM-COVENANT-114-NNN-PROPAGATION)** — *por qué*: un pacto solo es universal si está verificablemente presente en *todos* los consumidores, no solo en el repositorio insignia; *cómo*: una búsqueda literal del número de cláusula en los consumidores, respaldada por una metaprueba del §1.1 que demuestra que la verificación de propagación puede fallar.
- **submodules-catalogue.md (§11.4.74)** — *por qué*: la forma más rápida de violar la disciplina anti-duplicación es no saber lo que ya se posee; *cómo*: un inventario de 142 repositorios agrupados por capacidades, con una verificación del catálogo registrada en el rastreador *antes* de crear cualquier nuevo andamiaje.
- **Exportación multiformato** — *por qué*: la misma ley debe ser igualmente consumible por humanos que la leen, herramientas que la analizan y archivos que la preservan; *cómo*: cada documento canónico se emite como .md / .html / .pdf / .docx a partir de una única fuente.

Notas sobre estado y honestidad

- **Estado: enviado.** Versión activa en uso como submódulo en toda la flota (repositorios públicos canónicos y espejo).
- **Licencia: por determinar** — no especificada explícitamente en el material fuente revisado; confirmar con el archivo LICENSE del repositorio antes de publicar.
- Espejos adicionales en upstream: GitLab helixdevelopment1/helixconstitution, GitFlic helixdevelopment/helixconstitution, GitVerse helixdevelopment/HelixConstitution.

Nivel de prioridad: Helix-principal — pilar de gobernanza obligatorio en la construcción de todo lo perteneciente a la familia Helix.